

MINISTRY OF EDUCATION AND SCIENCE OF UKRAINE
IVAN FRANKO NATIONAL UNIVERSITY OF LVIV

Faculty of Applied Mathematics and Informatics
Department of Discrete Analysis and Intelligent Systems

COURSEWORK

Forecasting the Efficiency of Logistics Operations Using Machine Learning

Completed by: student of group PMI-33s
Specialty 122 “Computer Science”
Lukianchuk Denys Yevheniiiovych

Academic supervisor:
Associate Professor
Baranov Mykola Viktorovych

National grade _____
Number of points: _____ ECTS grade: _____

Contents

INTRODUCTION	2
1 THEORETICAL AND MATHEMATICAL FOUNDATIONS	4
1.1 The Problem of Forecasting in Logistics	4
1.2 Formulation of the Problem as a Binary Classification Task	4
1.3 Comparative Analysis of Machine Learning Methods	5
1.4 The Mathematical Principle of XGBoost	6
1.5 Review of Related Studies	7
1.6 Metric System for Objective Evaluation	7
2 SOFTWARE IMPLEMENTATION AND ALGORITHMIC SUPPORT	9
2.1 System Architecture	9
2.2 Input Data and the Identified Problem	10
2.3 Country Name Normalization Algorithm	10
2.4 Geocoding and Distance Calculation	11
2.5 Hybrid Weather Model	12
2.6 Formula for Realistic Profitability	13
2.7 Feature Engineering	13
3 EXPERIMENTAL STUDIES AND ANALYSIS OF RESULTS	15
3.1 Dataset and Sample Split	15
3.2 Cross-Validation	15
3.3 Hyperparameter Optimization with Optuna	16
3.4 Results of the Final Model	16
3.5 Analysis of ROC and Precision-Recall	18
3.6 Classification Threshold Optimization	19
3.7 Interpretation Using SHAP	21
3.8 Example of System Operation	24
CONCLUSIONS	25
References	27

INTRODUCTION

Relevance of the topic. Today, logistics is one of the foundations of the global economy. However, working in this field is becoming increasingly difficult: supply chains are becoming more complex, more modes of transport and intermediaries are involved, and regulations in different countries are constantly changing. It is no longer enough simply to deliver cargo — it is necessary to understand in advance whether a logistics operation will be profitable. The problem is that accurately forecasting profit is very difficult: data may be incomplete, while the external environment is often unpredictable.

Machine learning provides a powerful tool for working with such data. Modern algorithms, particularly gradient boosting, are capable of identifying relationships between hundreds of parameters that a person would not be able to detect manually. A system built on such algorithms allows companies not only to correct mistakes after they occur, but also to predict risks before signing a contract.

The aim of the work is to develop and study a system for classifying logistics operations as profitable or unprofitable using the XGBoost algorithm and integrating external data, including weather, fuel, and macroeconomic characteristics of destination countries.

The objectives of the research are:

1. To analyze approaches to forecasting in logistics and compare machine learning methods for tabular data;
2. To formulate the task of profitability assessment as a binary classification problem;
3. To develop a modular system architecture with the integration of external and local reference data such as geocoding, weather, and macroeconomic data;
4. To implement an algorithm for normalizing textual geographic data in order to eliminate language inconsistencies in the dataset;
5. To design a formula for calculating realistic profitability with dynamic penalties;

6. To train and optimize the XGBoost model and provide a technical interpretation of the results.

The object of the research is the processes of automated assessment of the financial efficiency of logistics operations.

The subject of the research is gradient boosting methods, feature engineering algorithms, and approaches to integrating external data into an ML system.

Practical significance. The developed system can serve as a basis for a decision support module in logistics companies. It allows users to obtain a forecast and understand the factors that may cause an operation to be unprofitable.

CHAPTER 1. THEORETICAL AND MATHEMATICAL FOUNDATIONS

1.1 The Problem of Forecasting in Logistics

Logistics is one of the most data-intensive industries, where the profitability of each individual order depends on dozens of factors simultaneously. The classical approach of building a tariff grid based on distance and cargo weight no longer reflects reality. The price of diesel fuel may increase by 20% within a week, an unexpected heavy rain may delay air cargo for a full day, and the customs rate may change depending on the current LPI of the destination country [15].

In the scientific literature, the application of machine learning in logistics covers a wide range of tasks: from forecasting delivery delays to optimizing costs and profit. Studies [6, 7] show that ensemble and boosting approaches are promising for the analysis of real logistics data, especially when dealing with complex nonlinear dependencies and a large number of factors. Similar results regarding the high efficiency of XGBoost are also presented in [8], where this algorithm was used to analyze supply chain data and forecast their operational efficiency.

1.2 Formulation of the Problem as a Binary Classification Task

From the perspective of computer science, the considered problem is a binary classification task. This task can be solved using supervised learning. We have a set of examples $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where $\mathbf{x}_i$ is the feature vector of the i -th order, and $y_i \in \{0, 1\}$ is the class label:

- $y = 1$ (Profitable) — the revenue from the order exceeds all real costs such as fuel, weather risks, and customs expenses;
- $y = 0$ (Unprofitable) — the order does not cover operational costs.

It is important that the label y does not directly correspond to the initial financial result indicators from the dataset. It is calculated using a formula with dynamic penalties, which is described in Subsection 2.6. This makes it possible

to form a more relevant target variable than a simple indicator of accounting profit.

The model learns to find a function $f : \mathbb{R}^M \rightarrow [0, 1]$ which, for a new order $\mathbf{x}$, returns the *probability* that it is profitable. The final decision is made according to a threshold τ :

$$\hat{y} = \begin{cases} 1, & f(\mathbf{x}) \geq \tau \\ 0, & f(\mathbf{x}) < \tau \end{cases} \quad (1)$$

The choice of τ is a separate and important step, which will be discussed in detail in Chapter 3.

1.3 Comparative Analysis of Machine Learning Methods

There are several approaches to solving a binary classification problem on structured tabular data. Within the theoretical analysis, three main methods were considered:

Logistic regression [4]. This is the simplest option, in which the model searches for a linear boundary between classes in the feature space. It is characterized by high computational speed and clear interpretability, since the analysis of weight coefficients makes it possible to directly estimate the degree of influence of each parameter on the final result. However, a linear model is not able to capture complex nonlinear interactions between features, for example, when the profitability of a logistics operation sharply decreases specifically under the combination of a long route distance and unfavorable weather conditions.

Random Forest [4]. This is an ensemble method that builds a large number of independent decision trees on random subsets of data and features, and then averages their responses. This approach effectively handles nonlinear dependencies and is resistant to overfitting. The main disadvantages of the method are a significant slowdown of the training process on large datasets, as well as the need for mandatory preliminary encoding of a large number of categorical features.

Gradient boosting, in particular XGBoost [1]. Unlike the parallel training used in Random Forest, decision trees here are built sequentially, where each subsequent model minimizes the errors of the previous ones. According to

large-scale comparative studies on standard datasets [9], this approach demonstrates the highest efficiency and accuracy specifically on structured tables. An additional advantage of XGBoost is its built-in optimized support for categorical features, which is critically important for analyzing geographical names and delivery types in the current study.

The analysis of related scientific works and the theoretical characteristics of the algorithms shows that gradient boosting algorithms are the most appropriate choice for structured logistics data with numerical and categorical features. Therefore, within this work, the XGBoost algorithm was selected as the main tool for implementation and research.

1.4 The Mathematical Principle of XGBoost

XGBoost [1] builds an ensemble of K decision trees. The prediction for the i -th example is the sum of the outputs of all trees:

$$\hat{y}_i = \sum_{k=1}^K f_k(\mathbf{x}_i) \quad (2)$$

At each step t , the algorithm adds a new tree f_t that minimizes the objective function:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \ell(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) \quad (3)$$

where ℓ is the loss function. For binary classification, this is logistic loss. $\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \|w\|^2$ is the regularization term that penalizes tree complexity, where T is the number of leaves and w denotes their weights.

To efficiently minimize (3), the algorithm uses a second-order Taylor approximation. For each example, the following values are computed:

$$g_i = \frac{\partial \ell(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}, \quad h_i = \frac{\partial^2 \ell(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2} \quad (4)$$

The optimal weight for leaf j is then calculated analytically:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \quad (5)$$

where I_j is the set of examples that fall into leaf j . This is the key advantage of the XGBoost algorithm: instead of performing full gradient descent, it finds the analytical optimum for each leaf in a single iteration.

1.5 Review of Related Studies

In [6], the authors investigated the capabilities of the XGBoost algorithm for classifying order delays in the field of e-commerce using a dataset containing more than 96 thousand records. The model achieved an overall accuracy of 93.24%; however, the authors point out a significant class imbalance problem: the recall for delayed orders was close to zero, which is a typical difficulty in binary classification tasks with an uneven distribution of labels. The study [7] applies machine learning methods, including XGBoost and recurrent neural networks, to demand forecasting and risk management in supply chains, introducing comprehensive evaluation metrics such as CAE and CAE-ESG, which go beyond standard accuracy.

Special attention should be paid to [8], where optimized XGBoost is applied to forecast the financial and operational efficiency, namely profitability, of supply chains in the context of Industry 4.0. However, most existing publications are limited to internal data from logistics systems and do not take into account the dynamics of the external environment, such as weather, fuel prices, and macroeconomic indicators.

1.6 Metric System for Objective Evaluation

To ensure robustness to class imbalance, the use of the standard accuracy metric may create a misleading impression of model quality. For example, if a baseline model always predicts the most common class, namely “profitable”, it will automatically demonstrate a high overall accuracy score, while failing to identify any unprofitable orders. Since unprofitable operations carry the main financial risk for a logistics company, it is necessary to use alternative evaluation metrics.

Therefore, a set of metrics robust to imbalance was used [4]:

- **ROC AUC** shows how well the model distinguishes profitable orders from unprofitable ones independently of the classification threshold. A value of 0.5 corresponds to random guessing, while 1.0 corresponds to a perfect model.
- **Macro F1-score** is a generalizing metric based on precision and recall:
 - **Precision** defines the proportion of correct predictions among all objects assigned by the algorithm to a particular class: $Precision = \frac{TP}{TP+FP}$.
 - **Recall** reflects the model’s ability to identify all true objects of this class: $Recall = \frac{TP}{TP+FN}$.

Macro F1-score is calculated as the harmonic mean of these two indicators separately for each class. This is critically important because we are equally interested in the quality of detecting both profitable and unprofitable shipments.

- **MCC (Matthews Correlation Coefficient)** is considered one of the most objective metrics for binary classification under class imbalance [5]. It takes into account all four types of prediction outcomes simultaneously:

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

where TP , TN , FP , and FN denote true positive, true negative, false positive, and false negative results.

- **Brier Score** is the mean squared error between the predicted probability and the true label. It evaluates not only the correctness of the predicted class, but also the quality, or calibration, of the predicted probabilities. The lower the score, the better.
- **Average Precision (AP)** is the area under the Precision-Recall curve. It is especially important under class imbalance because it is highly sensitive to the quality of detecting the rarer class, which in the considered case is unprofitable orders.

CHAPTER 2. SOFTWARE IMPLEMENTATION AND ALGORITHMIC SUPPORT

2.1 System Architecture

The main feature and scientific-practical novelty of the proposed approach lies in the comprehensive enrichment of the basic dataset (Data Enrichment). Unlike existing solutions, the architecture of the developed system provides for the integration of external factors: weather features, oil prices, and macroeconomic indicators such as GDP and the LPI index of destination countries. In addition, an important authorial contribution is the implementation of a specialized preprocessing stage for correcting systematic encoding errors in Spanish city names in the DataCo dataset, which significantly improved the quality of the model’s spatial features.

When designing the software itself, a modular architecture based on the principle of *Separation of Concerns* was chosen: each module is responsible for one specific task and nothing more. The project is written in Python 3.11 and consists of seven modules¹.

Table 1 – Structure of the software system

Module	Purpose
<code>config.py</code>	All constants, country name normalization dictionaries, macroeconomic data, and customs rates
<code>geo_service.py</code>	Geocoding via the Nominatim API, distance calculation using the haversine formula, and result caching
<code>weather_service.py</code>	Obtaining weather data from the Open-Meteo API or offline approximation
<code>economics_service.py</code>	GDP PPP, LPI, Brent prices, and customs duty calculation
<code>pipeline.py</code>	Main ETL pipeline: from raw CSV files to prepared features
<code>train_model.py</code>	XGBoost training, optimization with Optuna, SHAP, and metrics
<code>main.py</code>	CLI interface: pipeline execution, training, and manual prediction

The system supports a special offline mode using the environment variable

¹The full source code of the system is available in the public repository: <https://github.com/Delukich/Coursework.git>

LOGISTICS_OFFLINE=1. In this mode, instead of direct API requests, local caches, multilevel fallback mechanisms, and mathematical approximations are used. This makes it possible to form a complete feature vector even when access to external services is unavailable.

2.2 Input Data and the Identified Problem

The **DataCo Smart Supply Chain** dataset [14] was used to train the model. It contains 180 519 records of real logistics operations of a company for the period 2015–2018 and includes 53 input columns. Such a significant amount of data made it possible to successfully train the XGBoost algorithm and identify stable patterns in logistics processes.

During the initial analysis, a nontrivial problem was identified: all country names in the dataset were written in Spanish. For example: **Francia**, **Alemania**, **Países Bajos**. This is critical because the economics module searches for data using English country names. Due to this language mismatch, a significant share of orders incorrectly received the default value $GDP = 15\,000$ instead of the real indicator of the corresponding market zone. For example, for **Alemania** (Germany), the base value $GDP = 15\,000$ was assigned instead of the actual value of 52 000, which created a substantial distortion of the input parameters.

2.3 Country Name Normalization Algorithm

To solve this problem, the function `normalize_country_name()` was implemented in the file `config.py`. It works in four steps:

1. **Direct translation** using the `SPANISH_TO_ENGLISH` dictionary with more than 90 entries. This is the fastest step — an $O(1)$ lookup. It covers the most frequent cases.
2. **Fixed abbreviations** using `QUICK_FIX` for example, “usa” $\rightarrow$ “USA”, “uk” $\rightarrow$ “UK”.
3. **Fuzzy search using the pycountry library** — a standardized ISO 3166 country database.
4. **The Levenshtein algorithm** via `fuzzywuzzy` with a similarity thresh-

old of 92% for the main search and 96% for fallback cases. This helps process spelling errors.

As a result of applying the developed algorithm, the share of orders for which default macroeconomic indicators were used was minimized to a critically low level. For the vast majority of countries, this made it possible to successfully correct the model’s input features and ensure their connection to the real economic context:

Table 2 – Examples of GDP correction after normalization

In the dataset	After normalization	GDP before	GDP after
Francia	France	15 000	45 300
Alemania	Germany	15 000	52 000
Reino Unido	UK	15 000	43 000
Países Bajos	Netherlands	15 000	61 000
Japón	Japan	15 000	48 000

2.4 Geocoding and Distance Calculation

To calculate the distance from the warehouse to the destination point, it is necessary to determine the coordinates of the delivery city. This geocoding process was implemented using the Nominatim API from OpenStreetMap [13].

In order to ensure high data processing speed and optimize interaction with network services when processing more than 180 thousand orders, a two-level caching mechanism was implemented in the system:

1. **The `@lru_cache(maxsize=20000)` decorator** provides instant access to results in RAM while the script is running.
2. **The persistent file `geocode_cache.json`** stores results between program runs. Writing to the file is implemented atomically through a temporary file and `os.replace`, in order to avoid data corruption in case of unexpected failures.

Thanks to this, when the pipeline is run again, geocoding several thousand unique “city–country” pairs is performed much faster than with a complete download from scratch, since most results are reused from the cache.

A multilevel coordinate determination approach is used to estimate distance. First, the system attempts to obtain city-level coordinates. If exact geocoding is unavailable, it applies coordinates at the country, region, or market level. This approach made it possible to almost completely eliminate the use of hard default values: after applying the new pipeline, the share of records with a fixed distance of 5000 km was reduced to a minimum.

The distance between the warehouse and the delivery city is calculated using the haversine formula [10], which takes into account the spherical shape of the Earth. For transcontinental routes, this is critically important, since a standard Euclidean metric would produce an error:

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (6)$$

$$d = 2 \cdot R \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right), \quad R = 6371 \text{ km} \quad (7)$$

2.5 Hybrid Weather Model

The system architecture supports the use of the Open-Meteo API [12], which can return historical temperature and precipitation values for specified coordinates and a given date. However, in the reproducible offline environment used for the final experiment, weather features were mainly formed using mathematical approximation and fallback mechanisms.

The method `_estimate_climate()` approximates temperature through a trigonometric model of seasonal cycles:

$$T(\varphi, M) = T_{\text{mean}}(\varphi) + A(\varphi) \cdot \cos\left(\frac{2\pi(M - M_{\text{peak}})}{12}\right) \quad (8)$$

where $T_{\text{mean}} = \max(-2, 28 - 0.42|\varphi|)$, $A = \min(18, 2 + 0.22|\varphi|)$, $M_{\text{peak}} = 7$ for the Northern Hemisphere and $M_{\text{peak}} = 1$ for the Southern Hemisphere, while M denotes the month number.

A similar logic with adapted coefficients depending on the climate zone ($|\varphi| < 15^\circ$, $15^\circ\text{--}30^\circ$, $> 30^\circ$) is used to estimate precipitation. As a result, the system forms adequate approximate weather estimates even without direct network access to historical data. The implementation of this hybrid model made it possible to significantly improve the quality of weather features: the

mass default temperature value (15.0 °C) disappeared, and the share of records with zero precipitation as a default value was reduced to a minimum.

2.6 Formula for Realistic Profitability

The “Profit” field was not taken directly from the dataset because it does not account for real logistics costs. Instead, a formula was developed that recalculates profit using dynamic penalties:

$$P_{\text{real}} = P_{\text{base}} - C_{\text{dist}} - C_{\text{weather}} - C_{\text{tariff}} \quad (9)$$

Each component is defined as follows:

$$C_{\text{dist}} = \frac{d}{1000} \cdot \frac{\text{fuel}}{60} \cdot k_{\text{ship}} \quad (10)$$

$$C_{\text{weather}} = \begin{cases} 2.0, & \text{rain} > 5 \text{ mm} \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

$$C_{\text{tariff}} = \text{total} \cdot r_{\text{cat}} \cdot \left(1 + \max\left(0, \frac{3 - \text{LPI}}{50}\right) \right) \quad (12)$$

where $k_{\text{ship}} \in \{1.0, 1.2, 2.0, 3.0\}$ is the delivery mode coefficient (Standard / Second / First / Same Day), r_{cat} is the base customs rate for the product category, and the LPI correction increases the tariff for countries with poor logistics infrastructure. If $P_{\text{real}} \geq 0$, then `Is_Profitable` = 1; otherwise, it is 0.

2.7 Feature Engineering

In addition to the raw fields of the dataset, a number of new features were constructed. The main idea is that *the model should receive already processed information rather than calculate feature interactions on its own*. This speeds up training and improves accuracy.

Special attention should be paid to `ship_sla_rank`. Initially, the system included the feature `shipping_speed_ratio` = `sched_days` / (`dist` / 500 + 1), which divided the scheduled delivery days by the “theoretical duration at 500 km/day”. However, the analysis showed that the field `Days for shipment (scheduled)` has only four unique values (0, 1, 2, 4) and corresponds 100% to

the delivery type. It is simply an **SLA class code**, not the real number of days. Dividing a class code by distance is meaningless, and most rows received values below 1.0, which made the feature almost constant. It was replaced with `ship_sla_rank`, which improved the quality of the model.

All calculations are implemented using `df.map()` and NumPy vectorization [11] — without `for` loops. This is important for performance: processing 180 thousand rows takes minutes rather than hours.

Table 3 – New engineered features of the system

Feature	Meaning	Type
<code>price_per_km</code>	Ratio of order amount to distance. A key indicator of route marginality	Numerical
<code>fuel_x_distance</code>	Product of oil price and distance / 1000. Estimate of fuel load	Numerical
<code>ship_sla_rank</code>	SLA class: Same Day=0, First=1, Second=2, Standard=3. Replaced the invalid <code>shipping_speed_ratio</code> feature	Numerical
<code>ship_cost_per_km</code>	Delivery mode coefficient $\times$ fuel / distance	Numerical
<code>discount_x_qty</code>	Discount $\times$ number of units	Numerical
<code>gdp_x_lpi</code>	GDP PPP $\times$ country LPI. Shows the “quality” of the market	Numerical
<code>inflation_risk</code>	1 if the country’s inflation is greater than 5%	Binary
<code>is_cold</code>	1 if temperature is below 0°C	Binary
<code>is_heavy_rain</code>	1 if precipitation is greater than 10 mm	Binary
<code>dest_lat_abs</code>	latitude of the destination point	Numerical
<code>cross_hemisphere</code>	1 if the warehouse and destination are in different hemispheres	Binary
<code>order_quarter</code>	Order quarter (1–4)	Numerical

CHAPTER 3. EXPERIMENTAL STUDIES AND ANALYSIS OF RESULTS

3.1 Dataset and Sample Split

After passing through the pipeline (`pipeline.py`), a dataset with **180 519 rows** and **38 features** was obtained, along with the target variable `Is_Profitable`. The class balance is as follows: 65.05% profitable orders (class 1) and 34.95% unprofitable orders (class 0).

A critically important step is the correct splitting of the data. In this work, a **three-part split** was used to avoid data leakage — a situation in which test data somehow influence the training process:

- **Train (70%, 126 358 records)** — the model is trained on this data, and only this part is used to calculate medians for filling missing values.
- **Validation (15%, 27 083 records)** — this subset is used with Optuna for hyperparameter tuning and for selecting the optimal classification threshold τ .
- **Test (15%, 27 078 records)** — this subset is held out from the beginning and is used for the final evaluation of the model quality.

The final model is retrained on the **Dev set = Train + Validation** in order to make maximum use of the available data. The split was performed in a stratified manner, meaning that each subset preserves the same class balance as the full dataset.

3.2 Cross-Validation

Before hyperparameter optimization, **5-fold cross-validation** was performed on the Dev set to evaluate the stability of the baseline model. The results are presented in Table 4.

The small standard deviation, with $\text{std} = 0.0025$ for AUC, indicates that the model is stable: it is not tuned to a specific subsample, but has learned general patterns in the data.

Table 4 – Results of StratifiedKFold CV (5 folds)

Fold	AUC	Accuracy	Macro F1
1	0.6560	0.6949	0.5841
2	0.6620	0.6961	0.5876
3	0.6546	0.6922	0.5807
4	0.6578	0.6968	0.5880
5	0.6563	0.6960	0.5849
Mean $\pm$ std	0.6573 $\pm$ 0.0025	0.6952 $\pm$ 0.0016	0.5850 $\pm$ 0.0026

3.3 Hyperparameter Optimization with Optuna

Instead of manually searching through parameters, the Optuna framework [3] was used. It implements the TPE algorithm (Tree-structured Parzen Estimator), a Bayesian approach in which each new trial is built taking into account the results of previous trials rather than being selected randomly. This is much more efficient than Grid Search: in 50 iterations, Optuna can find a configuration that Grid Search would require thousands of checks to discover.

Optuna searched over the following parameter space: `n_estimators`: 100–600, `max_depth`: 3–10, `learning_rate`: 0.005–0.2 on a logarithmic scale, `subsample`: 0.5–1.0, `colsample_bytree`: 0.5–1.0, `min_child_weight`: 1–15, `gamma`: 0–2, `reg_alpha` (L1): 0–1, and `reg_lambda` (L2): 0.5–3.

The best result was obtained in trial #49, where the validation ROC AUC reached 0.6716. The evaluation was performed specifically on the Validation set, while the Test set remained held out until the final model evaluation stage.

3.4 Results of the Final Model

After Optuna found the best hyperparameter configuration, the model was retrained on the full Dev set (Train + Validation) and then evaluated on the held-out test set. For a correct comparison, a naive prevalence baseline was also used: each order was assigned the same profitability probability, equal to the share of profitable orders in the test set ($p = 0.6505$). Since this value exceeds the standard threshold of 0.5, such a baseline effectively classifies all orders as profitable. The results of the final evaluation are shown in Table 5.

As can be seen from Table 5, the proposed model outperforms the naive baseline across all key criteria. This means that the model does not merely

Table 5 – Comparison of the final model with the naive prevalence baseline on the test set

Metric	Proposed model	Naive baseline
Accuracy	0.6746	0.6505
Balanced Accuracy	0.6177	0.5000
ROC AUC	0.6737	0.5000
Average Precision	0.7630	0.6505
MCC	0.2513	0.0000
Cohen Kappa	0.2476	0.0000
Brier Score	0.1999	0.2274
Macro F1	0.6214	0.3941

Naive baseline: each order is assigned the same profitability probability, equal to the share of profitable orders in the test set ($p = 0.6505$). After applying the standard threshold of 0.5, this approach classifies all orders as profitable.

reproduce the dominance of the more common class, but actually learns to separate profitable logistics operations from unprofitable ones.

Comprehensive performance evaluation. On the test set, the final model achieved Accuracy = 0.6746, Balanced Accuracy = 0.6177, ROC AUC = 0.6737, Average Precision = 0.7630, MCC = 0.2513, Cohen’s Kappa = 0.2476, and Brier Score = 0.1999. These results indicate a moderate but practically useful ability of the model to distinguish profitable orders from unprofitable ones, even under class imbalance.

Comparison with the baseline. The metrics robust to class imbalance are especially representative. Balanced Accuracy increased from 0.5000 to 0.6177, Macro F1 from 0.3941 to 0.6214, while MCC and Cohen’s Kappa increased from 0 to 0.2513 and 0.2476, respectively. This confirms that the model has real discriminative ability, whereas the naive baseline is effectively unable to distinguish one class from another.

Class-level analysis. For the unprofitable class, the model achieved precision of 0.54, recall of 0.43, and an F1-score of 0.48. For the profitable class, it achieved precision of 0.72, recall of 0.81, and an F1-score of 0.76. Thus, the model recognizes profitable orders better; however, after threshold optimization, it also demonstrates an acceptable ability to detect risky, unprofitable operations.

Quality of probabilities. ROC AUC = 0.6737 compared with 0.5000

for the baseline means that the model performs meaningful ranking of orders by profitability probability rather than assigning the same score to all objects. Average Precision = 0.7630 also substantially exceeds the baseline value of 0.6505, indicating better ranking quality for profitable orders.

Forecast calibration. The Brier Score of the final model is 0.1999, which is better than the baseline value of 0.2274. This means that the predicted probabilities of the model are more accurate and better aligned with the actual outcomes. This conclusion is also confirmed by the calibration curve (Figure 1).

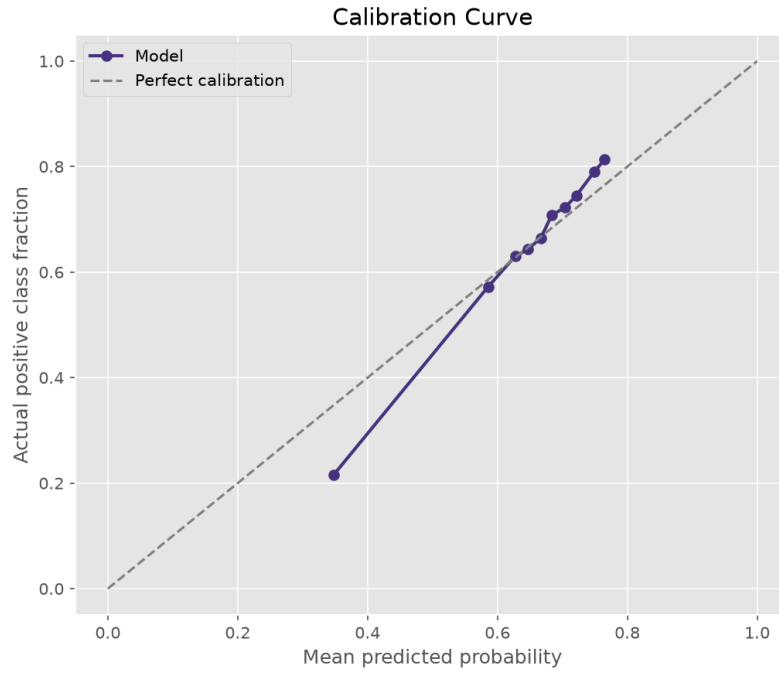


Figure 1 – Model calibration curve

3.5 Analysis of ROC and Precision-Recall

Figure 2 shows the ROC curve. $AUC = 0.6736$ means that if one profitable and one unprofitable order are selected at random, the model is likely to assign a higher score to the profitable one. The proposed approach, based on enriching the feature vector with external data, provides a stable and practically useful result for the task of separating profitable operations from unprofitable ones.

The Precision-Recall curve (Figure 3) shows Average Precision = 0.7628 with a baseline level of 0.657. This means that the model ranks profitable orders significantly higher than would be expected from a random distribution.

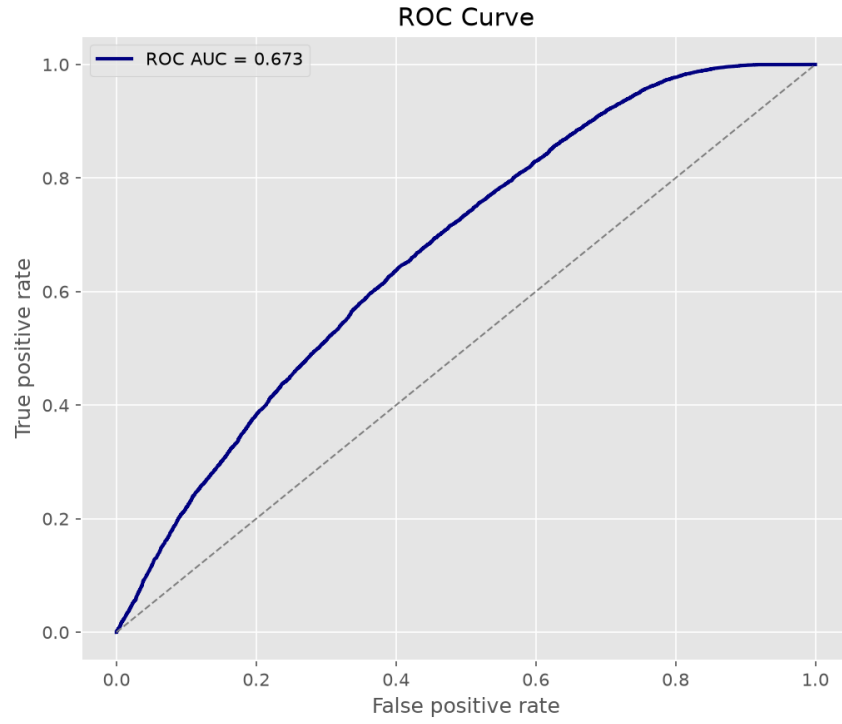


Figure 2 – ROC curve of the model (AUC = 0.6736)

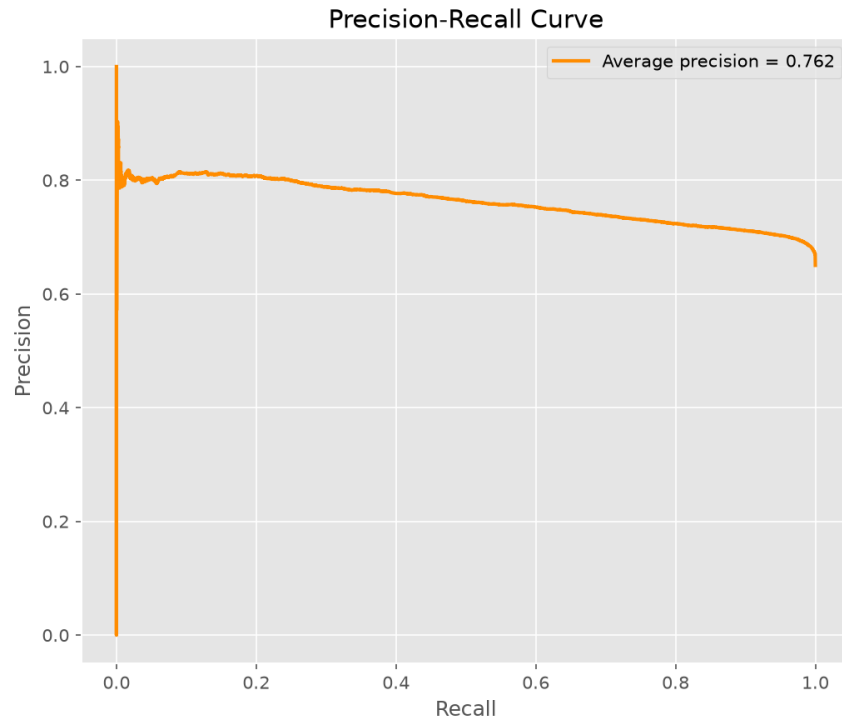


Figure 3 – Precision-Recall curve (AP = 0.7628)

3.6 Classification Threshold Optimization

By default, the threshold is equal to 0.5, but this is not always optimal. A search was performed over thresholds $\tau \in [0.05, 0.95]$ with a step of 0.01 on

the **Validation set**, maximizing Macro F1. The result is shown in Figure 4.

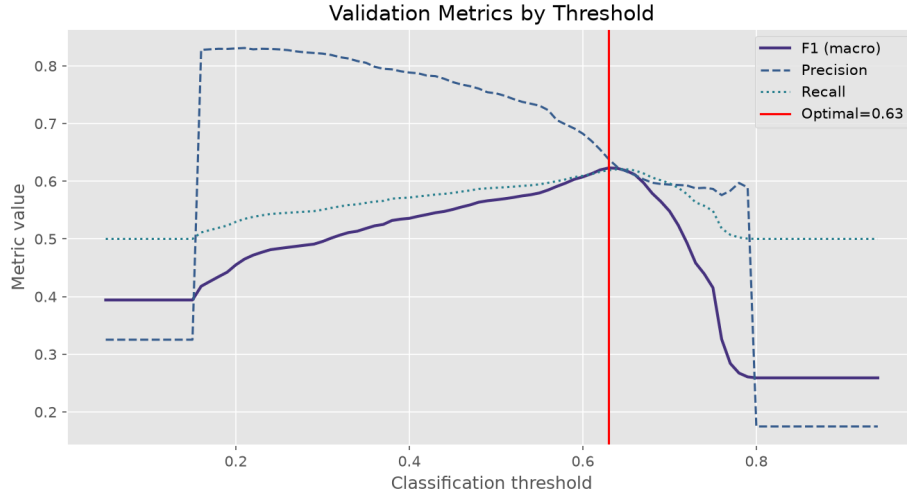


Figure 4 – Dependence of Macro F1 on the classification threshold

For a deeper understanding of why the standard threshold of 0.5 is inefficient for the current task, it is useful to analyze the distribution of predicted probabilities for both classes (Figure 5).

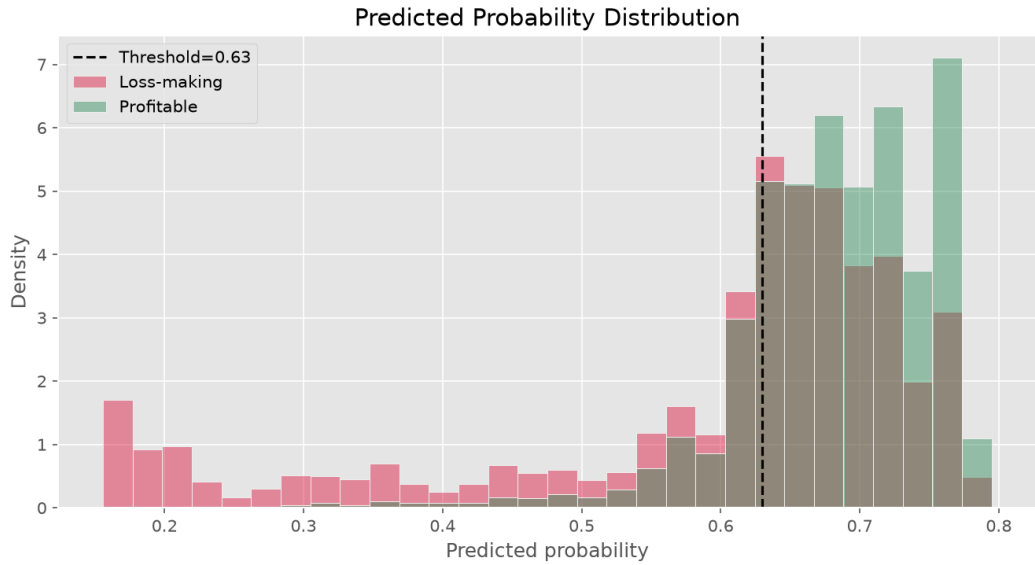


Figure 5 – Distribution of predicted probabilities for profitable and unprofitable orders

As can be seen from the histogram, the classes significantly overlap in the probability range from 0.5 to 0.7. Many actually unprofitable orders, shown by pink bars, receive a model score above 0.5, which under the standard threshold would classify them as “profitable” false positives. The optimal threshold $\tau = 0.63$, marked by the black dashed line, passes exactly through the point

where the density of profitable orders begins to significantly dominate over unprofitable ones.

The optimal threshold was found to be $\tau = 0.63$. At this threshold, Macro F1 = 0.6229 on the Validation set. Increasing the threshold to this value allows the recall of the unprofitable class to remain at approximately 0.43, meaning that the model detects a substantial share of risky orders without reducing the task to overly conservative rejections. The cost of this is a certain number of profitable orders being rejected. However, in logistics, accepting an unprofitable order is much more costly than rejecting a profitable one, so this compromise is justified.

The result at $\tau = 0.63$ on the Test set is shown in the confusion matrix (Figure 6).

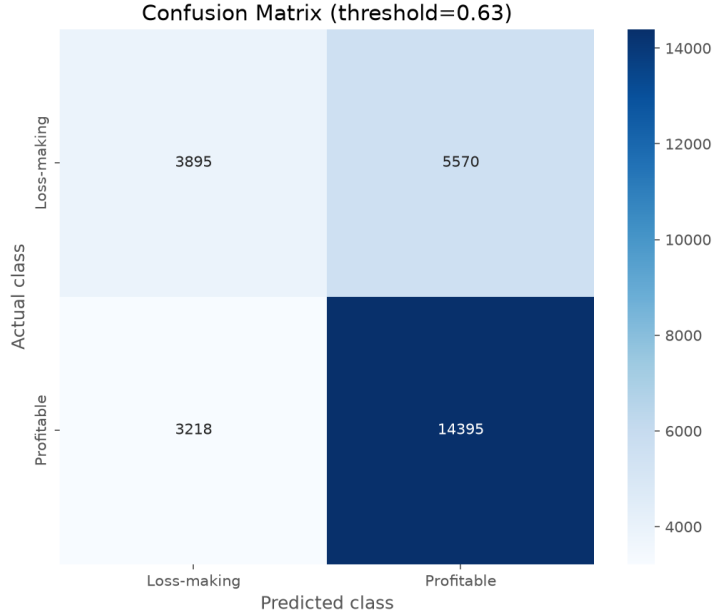


Figure 6 – Confusion matrix at threshold 0.63

3.7 Interpretation Using SHAP

One of the problems of gradient boosting is that it is difficult to explain *why* the model made a particular decision. To address this, **SHAP** [2] was used. This method is based on game theory, specifically Shapley values. The main idea is that for each order and each feature, SHAP calculates how much that feature shifted the model prediction relative to the average prediction.

First, it is useful to evaluate the general, or global, importance of the con-

structured features using the analysis of mean absolute SHAP values (Figure 7). This indicator demonstrates the absolute scale of each feature’s influence on the final model prediction, regardless of the direction of that influence.

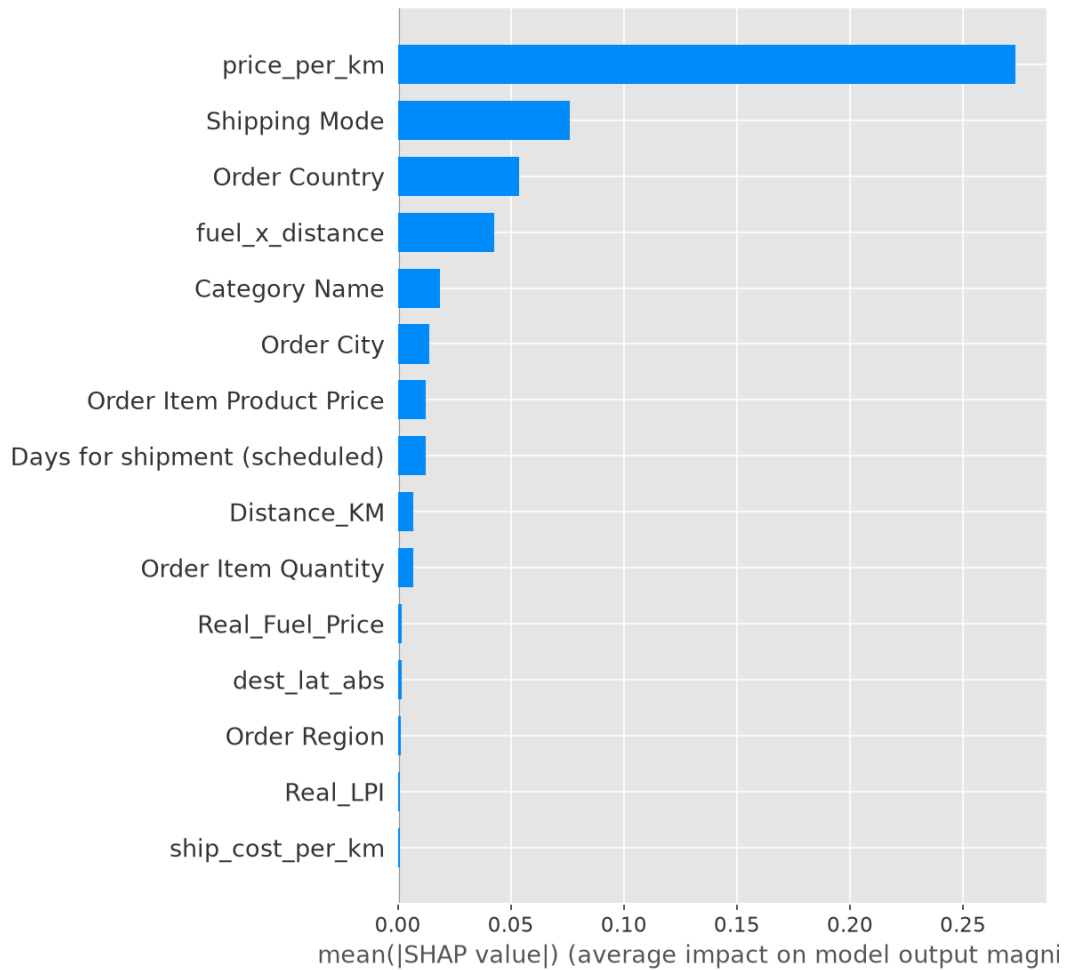


Figure 7 – Global feature importance by mean absolute SHAP value

As can be seen from the chart, the constructed feature **price_per_km** is the clear leader. Among the most important features are also those related to fuel load, delivery mode, and the spatial and geoeconomic context of the route. This distribution of importance confirms the effectiveness of the chosen methodology: the key role in forecasting is played specifically by engineered features built on the basis of the logistics meaning of the task.

The same tendency is confirmed by the standard feature importance based on the internal criteria of the XGBoost algorithm (Figure 8), namely the Gain criterion. The feature **price_per_km** expectedly became the leader here as well.

The Beeswarm plot is even more informative (Figure 9). Each point represents one order. The horizontal position of the point shows how much this

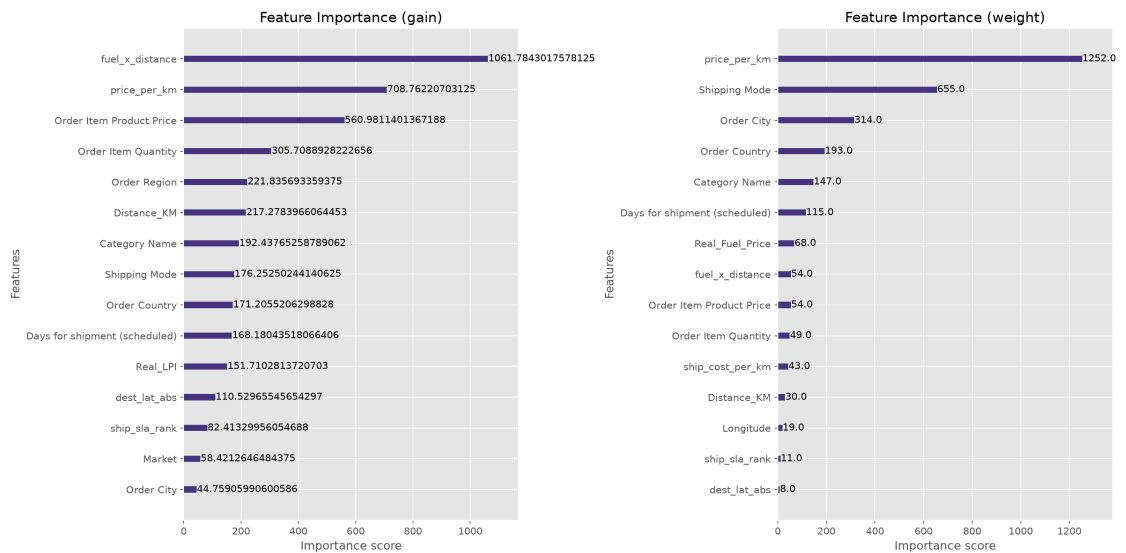


Figure 8 – Feature importance according to the Gain criterion

feature shifted the prediction toward “profitable” on the right or “unprofitable” on the left. The color shows the feature value: pink indicates a high value, while blue indicates a low value.



Figure 9 – SHAP Beeswarm plot: direction of feature influence

Key conclusions from SHAP:

- `price_per_km` — a high order price per unit of distance, represented by pink points, consistently pushes the prediction toward profitability.
- `fuel_x_distance` — high fuel costs, caused by long routes combined with expensive oil, push the prediction toward unprofitability.
- `Order Country` — the destination country has a strong influence: it is an aggregated signal of customs rates, LPI, and the market's GDP.

This analysis makes it possible to explain the model's logic in simple terms, which is critical for implementation in a real company: a manager wants to understand *why* the system recommends rejecting an order.

3.8 Example of System Operation

Finally, let us consider a specific example from the CLI mode: a route from a warehouse in New York to Paris, Standard Class, Electronics, 1 unit, \$500, with a 10% discount.

The system calculated:

- Distance: 5 834.74 km, according to the haversine formula;
- Fuel penalty: \$7.97, calculated as $\text{distance} \times \text{Brent}/60 \times \text{Standard coefficient}$;
- Customs duty: \$0.00, since Electronics shipped from the USA to France have a zero rate;
- Weather penalty: \$0.00, since precipitation is 1.4 mm, which is below the penalty threshold;
- Verdict: **PROFITABLE**, probability 73.40% with a threshold of 64.0%.

The result is logical: the product has a sufficiently high price relative to the distance ($\approx \$0.08/\text{km}$, which is a high value of `price_per_km`), the customs duty is absent, and the weather conditions do not create any additional penalty load.

CONCLUSIONS

In the course of this coursework, a system for classifying logistics operations as profitable or unprofitable was developed and studied. The system is based on the XGBoost algorithm and uses external weather, fuel, and macroeconomic characteristics. Based on the completed work, the following results can be formulated:

1. **The analysis of scientific literature and algorithm properties** showed that gradient boosting algorithms are an appropriate choice for structured tabular data with numerical and categorical features. Therefore, within this work, XGBoost was selected as the main algorithm for implementation and research.
2. **The problem of Spanish country names in DataCo was eliminated:** the developed four-level normalization algorithm significantly improved the quality of linking orders to the macroeconomic characteristics of countries and reduced the number of incorrect default values.
3. **A formula for realistic profitability was designed** with dynamic penalties for distance, weather conditions, and customs duties with LPI correction. This made it possible to move from the formal profit indicator in the dataset to a more meaningful assessment of the operational efficiency of an order.
4. **The feature generation pipeline was improved:** after implementing a multilevel fallback mechanism for coordinates and weather data, it became possible to eliminate the mass use of hard default values. In the final dataset, the use of the fixed distance value of 5000 km was reduced to a minimum, the default temperature of 15.0°C was completely eliminated, and the number of cases where zero precipitation was used as a default value became isolated.
5. **A three-part Train/Validation/Test split was implemented.** The Validation set was used for hyperparameter tuning and classification threshold optimization, while the Test set was used for the final evaluation of model quality.

6. **Improved model results were achieved:** on the test set, the final model achieved ROC AUC = 0.6737, Accuracy = 0.6746, Balanced Accuracy = 0.6177, Average Precision = 0.7630, MCC = 0.2513, Cohen’s Kappa = 0.2476, and Brier Score = 0.1999. Cross-validation also confirmed the stability of the model: the mean ROC AUC was 0.6573 with a standard deviation of 0.0025.
7. **Classification threshold optimization** showed that the best threshold is $\tau = 0.63$. At this threshold, Macro F1 on the validation set reached 0.6229, while the recall of the unprofitable class on the test set remained at approximately 0.43, which is practically important for detecting risky orders.
8. **SHAP analysis** confirmed that the most important predictor is `price_per_km`. Among the most influential features are also characteristics related to fuel load, delivery mode, and the geoeconomic context of the route. This makes the model’s decisions interpretable and suitable for practical use.

REFERENCES

- [1] **Chen T., Guestrin C.** XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD*. 2016. P. 785–794.
- [2] **Lundberg S. M., Lee S. I.** A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*. 2017. P. 4765–4774. URL: <https://arxiv.org/abs/1705.07874>
- [3] **Akiba T., Sano S., Yanase T., Ohta T., Koyama M.** Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD*. 2019. P. 2623–2631.
- [4] **Pedregosa F. et al.** Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*. 2011. Vol. 12. P. 2825–2830.
- [5] **Chicco D., Jurman G.** The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*. 2020. Vol. 21. P. 6.
- [6] **Lesmana S. P., Hermawati D., Mukaromah M. et al.** Machine Learning Implementation for E-commerce Delivery Delay Prediction Using XGBoost Algorithm. *Green Engineering: International Journal of Engineering and Applied Science*. 2025. Vol. 2, No. 3. P. 46–57.
- [7] **Sattar M. U., Dattana V., Hasan R. et al.** Enhancing Supply Chain Management: A Comparative Study of Machine Learning Techniques with Cost–Accuracy and ESG-Based Evaluation for Forecasting and Risk Mitigation. *Sustainability* (MDPI). 2025. Vol. 17, No. 13. P. 5772.
- [8] **Nsir N., Alzubi A. B., Adegboye O. R.** Enhancing Sustainable Supply Chain Performance Prediction Using an Augmented Algorithm-Optimized XGBOOST in Industry 4.0 Contexts. *Sustainability* (MDPI). 2025. Vol. 17, No. 22. P. 10344. DOI: 10.3390/su172210344.
- [9] **Grinsztajn L., Oyallon E., Varoquaux G.** Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems*. 2022. Vol. 35. P. 507–520.

- [10] **Sinnott R. W.** Virtues of the Haversine. *Sky and Telescope*. 1984. Vol. 68, No. 2. P. 158–159.
- [11] **Harris C. R. et al.** Array programming with NumPy. *Nature*. 2020. Vol. 585. P. 357–362.
- [12] **Open-Meteo**. Historical Weather Data Documentation. Available at: <https://open-meteo.com/en/docs> (accessed: 15.04.2026).
- [13] **Nominatim API**. OpenStreetMap Geocoding Service. Available at: <https://nominatim.org> (accessed: 15.04.2026).
- [14] **Constante F., Silva F., Pereira A.** DataCo SMART SUPPLY CHAIN FOR BIG DATA ANALYSIS. *Mendeley Data*. 2019. V3. DOI: 10.17632/8gx2fvg2k6.3.
- [15] **World Bank**. Logistics Performance Index (LPI) Report. Available at: <https://lpi.worldbank.org> (accessed: 18.04.2026).
- [16] **GitHub repository of the project:**
<https://github.com/Delukich/Coursework.git> (accessed: 15.05.2026).